Documentación oficial de la base de datos de EduControl v1.0

Joan David Sáenz Botero & Nicolas Felipe Rueda Aguilar

Facultad de Ingeniería, Universidad de Cundinamarca

Ingeniería de Software II 602M

Ing. German Ortiz Diaz

15 de septiembre de 2026

Tabla de contenido

Introducción **5**

Diccionario de datos **6**

Tabla de usuarios 6

Tabla de herramientas 6

Tabla de proyectos 7

Tabla de prestamos 7

Cardinalidades 8

Análisis de normalización **9**

Primera norma formal (1NF) 9

Segunda norma formal (2NF) 10

Tercera norma formal (3NF) 12

Violaciones y defectos 14

Violación 3NF - *herramientas.disponible* es un atributo derivado 14

Violación 3NF - *prestamos.estado* es un atributo derivado 14

Violación 3NF (Modelo local) - *Prestamo.receptor* duplica *Usuario.nombre* 15

Observación del modelado - *herramientas.cantidad* impide la trazabilidad unitaria 15

Defecto de integridad – Ausencias de restricciones UNIQUE 16

Observación – Mezcla de identidad y autenticación en usuarios 16

Conclusiones 17

Tabla de tablas

Tabla 1. Tabla *usuarios* 6

Tabla 2. Tabla *herramientas* 6

Tabla 3. Tabla *proyectos* 7

Tabla 4. Tabla *prestamos* 7

Tabla 5. Tabla de dependencias parciales 11

Tabla 6. Tabla de resumen general 17

Tabla 7. Matriz de priorización de correciones 17

Introducción

El presente documento detalla el diseño, estructura y análisis de normalización de la base de datos de Edu Control, un sistema de préstamo de herramientas educativas. La arquitectura de datos está soportada por el motor PostgreSQL 16 y gestionada mediante el ORM Prisma 6. Debido a los requerimientos no funcionales del sistema (arquitectura *offline-first*), el modelado hace uso de identificadores globales únicos (UUID v4) generados en el cliente para prevenir colisiones durante la sincronización. Este informe evalúa el esquema actual bajo las reglas de la Primera, Segunda y Tercera Forma Normal (1NF, 2NF, 3NF), identificando aciertos estructurales, violaciones por atributos derivados y proponiendo las correcciones necesarias para garantizar la integridad de los datos.

Diccionario de datos

Tabla de usuarios

Tabla 1. Tabla *usuarios*

Atributo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador global generado en el cliente
nombre	String	___	Nombre completo del actor
rol	Rol (enum)	NOT NULL	ADMINISTRADOR / RECEPCION / USUARIO
contrasena	string?	NULL	Credencial de acceso (opcional)
created_at	DateTime	now()	Alta de registro
updated_at	DateTime	@updatedAt	Última modificación

Nota. Tabla de atributos de usuario

Tabla de herramientas

Tabla 2. Tabla *herramientas*

Atributo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador global
nombre	String	___	Denominación de la herramienta
categoria	String	default(“General”)	Clasificación (texto libre)
estado_fisico	String	default(“Bueno”)	Condición física (texto libre)
descripcion	string?	NULL	Detalle opcional
cantidad	Int	NOT NULL	Unidades en inventario
disponible	Boolean	default(true)	Indicador de disponibilidad
created_at / updated_at	DateTime	___	Auditoría

Nota. Tabla de atributos de herramientas

Tabla de proyectos

Tabla 3. Tabla *proyectos*

Atributo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador global
nombre	String	___	Nombre del proyecto académico
descripcion	String?	NULL	Objetivo del proyecto
created_at / updated_at	DateTime	___	Auditodía

Nota. Tabla de atributos de proyectos

Tabla de prestamos

Tabla 4. Tabla *prestamos*

Atributo	Tipo	Restricción	Descripción
id	UUID	PK	Identificador global Herramienta prestada
herramienta_id	UUID	FK→ herramientas.id, ON DELETE RESTRICT	Herramienta prestada
usuario_id	UUID	FK→ usuarios.id, ON DELETE RESTRICT	Quien recibe
proyecto_id	UUID?	FK→ proyectos.id, ON DELETE SET NULL	Proyecto asociado (opcional)
fecha_salida	DateTime	now()	Momento de entrega
fecha_devolucion_esperada	DateTime	NOT NULL	Compromiso de retorno
fecha_devolucion_real	DateTime?	NULL	Retorno efectivo

estado	EstadoPrestamo(enum)	default(prestado)	prestado / devuelto / atrasado
created_at	DateTime	now()	Auditoría

Nota. Tabla de atributos de prestamos

Cardinalidades

- usuarios 1 ──< prestamos (un usuario tiene N préstamos)
- herramientas 1 ──< prestamos (una herramienta participa en N préstamos históricos)
- proyectos 1 ──< prestamos (opcional: proyecto_id es NULLABLE)

Análisis de normalización

Primera norma formal (1NF)

Criterio: todos los atributos deben ser atómicos (indivisibles), no debe haber grupos repetitivos ni colecciones dentro de una celda, y cada tabla debe poseer una clave primaria que identifique unívocamente cada tupla.

Cumplimiento verificado:

1. **Atomicidad.** Ningún atributo almacena listas ni estructuras compuestas. Un contraejemplo de lo que se evitó deliberadamente sería registrar el préstamo así:

id	***receptor***	***herramientas_prestadas***
a1b2	Luis Perez	"Multimetro, taladro, kit Arduino"

Ese diseño violaría 1NF porque *herramientas_prestadas* es multivaluado. En Edu Control cada relación herramienta–usuario ocupa una tupla independiente en *prestamos*, y el vínculo se expresa mediante la clave foránea *herramienta_id*. Tres herramientas prestadas al mismo estudiante producen tres filas, no una celda con tres valores.

2. **Separación de fechas.** Se evitó el anti patrón de guardar *"2026-09-01 / 2026-09-15"* en una sola columna de texto. El esquema descompone el ciclo de vida temporal en tres atributos atómicos y tipados: *fecha_salida*, *fecha_devolucion_esperada* y *fecha_devolucion_real*. Esto permite consultas como WHERE *fecha_devolucion_esperada* < NOW() sin parsing de cadenas.

3. **Clave primaria universal.** Las cuatro entidades declaran *id String @id @default(uuid()) @db.Uuid*. La elección de UUID v4 sobre un entero autoincremental no es estética: responde al requerimiento RNF-02. En una arquitectura *offline-first*, tres tablets sin conexión generando préstamos simultáneamente con *SERIAL* producirían inevitablemente los ids 1, 2, 3 en cada dispositivo y colisionarían al sincronizar. El UUID se genera en el cliente antes de escribir en IndexedDB, garantizando unicidad global sin coordinación con el servidor.
4. **Dominios cerrados mediante numeraciones.** *Rol* y *EstadoPrestamo* son tipos *enum* de PostgreSQL. Esto refuerza la atomicidad semántica: el motor rechaza físicamente un valor como 'ADMIN' o 'prestado/atrasado', imposibilitando estados ambiguos o compuestos.

Conclusión 1NF: el esquema cumple la primera forman normal. Se documenta una observación de modelado en la §3.4 (atributo *cantidad*) que, si bien no constituye violación formal, limita la trazabilidad unitaria.

Segunda norma formal (2NF)

Criterio: la relación debe estar en 1NF y todo atributo no primo debe depender funcionalmente completa de la clave primaria. Las violaciones de 2NF son las dependencias parciales, que por definición solo pueden existir si la clave primaria es compuesta.

Cumplimiento verificado: El argumento central es estructural: las cuatro tablas de Edu Control tienen clave primaria simple (un único atributo *id* de tipo UUID). Al no existir ninguna

clave compuesta, no existe subconjunto propio de la clave del cual un atributo pudiera depender parcialmente. La 2NF se satisface por construcción.

Este punto merece detalle porque es el error clásico en sistemas de préstamo. Un diseño ingenuo modelaría la tabla intermedia con clave compuesta:

prestamos(PK: (herramienta_id, usuario_id) , nombre_herramienta, categoria, nombre_usuario, rol, fecha_salida)

Ese modelo incurre en tres dependencias parciales simultáneas:

Tabla 5. Tabla de dependencias parciales

Dependencia detectada	**Tipo**	**Problema**
herramienta_id → nombre_herramienta, categoria	Parcial (depende de parte de la PK)	Renombrar una herramienta exige actualizar N préstamos
usuario_id → nombre_usuario, rol	Parcial	El rol del usuario se replica en cada préstamo
(herramienta_id, usuario_id) → fecha_salida	Completa	Correcta, pero impide prestar dos veces la misma herramienta al mismo usuario

Nota. Tabla que muestra el tipo y problema de cada dependencia detectada

Edu Control evita las tres. *prestamos.id* es un UUID artificial e independiente, y los atributos descriptivos de las entidades relacionadas no se replican: solo se referencian por FK. La determinación es total:

prestamos.id → { herramienta_id, usuario_id, proyecto_id, fecha_salida,

fecha_devolucion_esperada, fecha_devolucion_real, estado, created_at }

Ventaja funcional adicional: al no usar (*herramienta_id, usuario_id*) como clave, el sistema soporta de forma natural el historial acumulado, el mismo estudiante puede prestar el mismo multímetro en marzo, mayo y octubre, generando tres tuplas distintas. Esto es indispensable para el reporte de "herramientas más solicitadas" exigido por RF-08.

Conclusión 2NF: el esquema cumple la segunda forma normal de manera trivial y robusta.

Tercera norma formal (3NF)

Criterio: la relación debe estar en 2NF y ningún atributo puede depender de otro atributo que no sea la clave.

Cumplimiento parcial. El esquema respeta 3NF en la eliminación de datos descriptivos redundantes, pero incurre en dos violaciones por atributos derivados. Se documentan primero los aciertos y luego las violaciones con su corrección.

Aciertos:

1. *prestamos* no almacena datos descriptivos de sus entidades relacionadas. No existen columnas *nombre_herramienta*, *categoria_herramienta*, *nombre_usuario* ni *nombre_proyecto*. Si existieran, tendríamos la cadena transitiva *prestamos.id → herramienta_id → nombre_herramienta*, con la anomalía de actualización

correspondiente. El esquema fuerza la resolución vía JOIN, manteniendo una única fuente de verdad por dato.

2. *proyectos* es una entidad de primer orden, no un atributo. La implementación de RF-05 podría haberse resuelto con una columna *prestamos.nombre_proyecto* VARCHAR. Eso habría generado la dependencia transitiva *prestamos.id* → *nombre_proyecto* → *descripcion_proyecto* y habría replicado el nombre del proyecto en cada uno de los préstamos asociados. La extracción a tabla propia con FK *proyecto_id* elimina la transitividad y permite que la descripción del proyecto se actualice en un solo lugar.
3. **Opcionalidad correctamente modelada.** *proyecto_id* es NULLABLE con ON DELETE SET NULL. Un préstamo sin proyecto académico es válido según RF-05 (“debe permitir asociar”, no “debe exigir”), y la política de borrado evita tanto tuplas huérfanas como la pérdida del registro histórico del préstamo.
4. **La auditoría depende de la clave, no de otros atributos.** *created_at* y *updated_at* son función directa de la identidad de la tupla, no de ningún atributo no primo.

Violaciones y defectos

Violación 3NF - *herramientas.disponible* es un atributo derivado

- **Dependencia transitiva detectada:** *herramientas.id → cantidad → disponible*.
- El atributo *disponible* no aporta información independiente, ya que es la evaluación lógica de *cantidad > 0*.
- **Anomalía materializada:** El formulario actualmente escribe *disponible: true* de forma incondicional. Un administrador podría fijar *cantidad = 0*, generando un estado inconsistente donde la herramienta aparece prestable sin stock real.
- **Corrección (Opción recomendada):** Conservar la columna, pero redefinirla semánticamente como *retirada* (Booleano). Esto indica una baja administrativa independiente del stock (ej. herramienta averiada), calculando la disponibilidad efectiva como *(cantidad > 0 AND retirada = false)*.

Violación 3NF - *prestamos.estado* es un atributo derivado

- **Dependencia transitiva detectada:** *prestamos.id → { fecha_devolucion_esperada, fecha_devolucion_real } → estado*.
- El valor del estado es calculable a partir de las fechas y el tiempo actual del sistema.
- **Anomalía materializada:** Existe una doble fuente de verdad. Un préstamo vencido en PostgreSQL permanece como 'prestado', haciendo que el panel de usuario muestre una etiqueta desactualizada, mientras que el módulo de reportes lo cuenta correctamente como moroso.

- **Corrección:** Limitar el enum *EstadoPrestamo* a hechos persistidos no derivables *(prestado*, *devuelto*, *cancelado*) y exponer la mora únicamente mediante una vista de base de datos (*v_prestamos_estado*) que calcule los días de retraso dinámicamente.

Violación 3NF (Modelo local) - *Prestamo.receptor* duplica *Usuario.nombre*

- Aunque el esquema de Prisma está limpio, el modelo en Dexie (Frontend) introduce redundancia guardando el nombre del receptor en la tabla de préstamos.
- **Agravante:** El backend consume este campo para materializar usuarios durante la sincronización, lo que puede generar identidades duplicadas (usuarios fantasma) en la nube.
- **Corrección:** Eliminar la propiedad *receptor* del modelo local de Dexie y resolver el nombre mediante una consulta (JOIN local) usando el *usuarioId*.

Observación del modelado - *herramientas.cantidad* impide la trazabilidad unitaria

- La tabla confunde el concepto de "tipo de herramienta" con "unidades físicas". Con un registro de *cantidad = 5*, es imposible saber exactamente cuál unidad está en mantenimiento o cuál unidad específica se prestó.
- **Corrección (Deuda técnica):** Separar el catálogo del inventario físico mediante dos tablas: *Herramienta* (el tipo) y *UnidadHerramienta* (el objeto físico individual con su propio estado).

Defecto de integridad – Ausencias de restricciones UNIQUE

- La base de datos carece de restricciones *UNIQUE*.
- Se está utilizando *usuarios.nombre* para el login y la deduplicación offline de forma insegura. Si existen dos usuarios con el mismo nombre, el sistema asume cualquiera de los dos, haciendo que el login no sea determinista.
- **Corrección:** Aplicar un índice único insensible a mayúsculas sobre la tabla usuarios: *CREATE UNIQUE INDEX usuarios_nombre_lower_key ON usuarios (LOWER(nombre));* .

Observación – Mezcla de identidad y autenticación en usuarios

- La columna *contrasena* es *NULLABLE* porque mezcla usuarios operativos (Admin/Recepción) con sujetos pasivos (Estudiantes). Esto impide agregar políticas de seguridad estrictas sin ensuciar la entidad base.
- **Corrección:** Separar la autenticación en una tabla independiente *Credencial* (relación 1:0..1), permitiendo almacenar hashes seguros (Bcrypt) aislados de los datos personales.

Conclusiones

Tabla 6. Tabla de resumen general

Forma normal	Estado	Fundamento
1NF	Cumple	Atributos atómicos, sin grupos repetitivos, PK UUID en las entidades y dominios cerrados por enum.
2NF	Cumple	Todas las PK son simples; no existen dependencias parciales.
3NF	Cumple parcialmente	Se identificaron 2 atributos derivados que generan dependencias transitivas (§3.1, §3.2) y redundancia local (§3.3).
BCNF	Condicionado	Se alcanzará al declarar *usuarios.nombre* como *UNIQUE* (§3.5).

Nota. Tabla que generaliza el contenido del presente documento

Tabla 7. Matriz de priorización de correciones

#	Hallazgo	Severidad	Impacto en código	Prioridad
§3.5	Sin *UNIQUE* en *nombre*	Crítica: login no determinista, identidades duplicadas al sincronizar	Bajo (migración + dedup)	1
§3.2	*estado* derivado	Alta: el panel y el reporte se contradicen	Medio	2

§3.1	*disponible* derivado	Alta: ya produce c*antidad:0 / disponible:true*	Bajo	3
§3.3	*receptor* redundante	Media: alimenta la creación de usuarios fantasma	Medio	4
§3.6	Identidad + autenticaciones acopladas	Media: bloquea el cifrado de credenciales	Alto	5
§3.4	*cantidad* sin trazabilidad unitaria	Media: limita RF-02 y RF-08	Alto	6
§3.7	*fecha_salida* / *created_at*	Baja: solo documental	Nulo	7

Nota. Tabla que indica las correcciones a realizar según su número

El esquema relacional de Edu Control presenta fundamentos sólidos, destacando la implementación de claves primarias UUID universales que satisfacen exitosamente las demandas de una arquitectura distribuida y offline-first. La extracción de proyectos como entidad independiente previene dependencias transitivas críticas. Las violaciones detectadas se limitan al uso de atributos derivados (disponible, estado), lo cual es un patrón común de desnormalización, y su corrección mediante vistas y columnas generadas asegura la consistencia de los datos. Se priorizará la resolución inmediata de la restricción UNIQUE en los nombres de usuario para garantizar el determinismo en la autenticación y prevenir la duplicación de identidades en la sincronización.